

PARISCV v2: A Validated Cycle Model for Custom-Instruction Discovery

Ashuthosh M. R.
Independent Researcher

Abstract—Choosing a custom instruction or an ISA extension for an embedded RISC-V core normally means writing RTL first and measuring afterwards. We present a trace-driven alternative: an analytical cycle model, derived from the OpenHW CORE-V user manuals rather than fitted to data, that consumes a Spike instruction trace and predicts real-core cycle counts within 0.25% worst case (typically under 0.01%) across seven Embench-IoT kernels on two cores, CV32E40X and CV32E40P. Because the model costs a trace rather than a simulation, it scores hardware that does not yet exist. We use it to predict enabling Zbb (within 2.7%) and the Xpulp extension (a measured $3.24\times$ speedup predicted to 0.10%), and to discover, add and verify four custom instructions whose measured savings all fell inside their predicted bands, recovering 10–15% of their kernels’ cycles where they paid at all. Measuring the CV-X-IF interface showed that an offload costs two cycles, which puts a floor of three cycles on any sequence worth offloading and pruned 33 nominally viable candidates to 11. A further 8.4% of cycles turned out to be recoverable with no hardware change whatsoever, lost purely to branch-target misalignment, which on one kernel exceeds the entire benefit of the bitmanip extension. All code, patches and measurements are open source.

I. Motivation

On a short in-order pipeline, arithmetic is rarely the bottleneck. Fig. 1 decomposes seven Embench-IoT [1] kernels on CV32E40X: branch and jump flushes take 14–26% of all cycles and IPC sits between 0.46 and 0.83. Most of the recoverable headroom is therefore not arithmetic at all.

Microarchitecture-aware instruction-set-extension (ISE) design for RISC-V has been studied before, reporting up

to $2.47\times$ on Embench and MiBench [2]. Our emphasis differs in two respects. First, the estimate here comes from a model of one specific target pipeline, taken from that core’s published documentation rather than fitted to results, and checked against the core’s own RTL before any of its recommendations are trusted. Second, we cost the mechanism by which a custom instruction attaches to the core, which turns out to decide which candidates can pay at all. The work continues PARISCV [9], a profiler for application-specific RISC-V optimisation, replacing its estimate with one a candidate can be scored against before any hardware for it exists. This paper reports that validation and what it then revealed.

II. Methodology

spike -l yields a commit trace of the unmodified binary. A Python model assigns each instruction its documented cost from the CV32E40X or CV32E40P user manual [3], [4] and adds documented penalties: taken-branch and jump flushes, branch-target misalignment, load-use hazards and jalr data hazards. Costs are documented, not fitted; the CV32E40P model subclasses the CV32E40X model and overrides only the differences the manuals state, so it doubles as an executable specification of how the two cores differ. Divide latency is data-dependent ($3 + \text{clz}(\text{divisor})$), so the model recovers each actual divisor by replaying register writes from spike -log-commits.

Code layout has to match. A taken branch costs three cycles, or four when its target is a non-word-aligned, non-compressed instruction, so two builds of identical C source linked at different addresses genuinely differ: we measured 5,844 against 6,356 cycles for the same algorithm. Each kernel is therefore built twice, Spike at a high address and RTL at the boot address, with .text verified byte-identical before any number is reported. Skip this and what looks like model error is mostly an artefact of the harness.

Candidate discovery runs on the same trace. The finder enumerates every straight-line window of up to four instructions, discards any window containing control flow, and canonicalises the rest by renaming registers to positional slots, so one computation collapses to a single candidate however it was allocated. A window survives only if it reads at most `X_NUM_RS` registers from the register file and leaves exactly one value live out, which is what one custom instruction can express; values both produced and consumed inside the window become

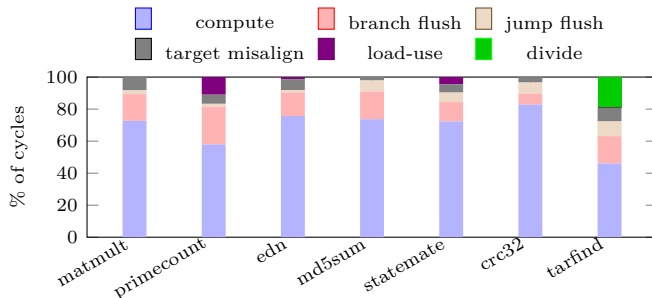


Fig. 1. Where the cycles go on CV32E40X. Compute is the IPC=1 ideal; everything above it is a stall the manual documents. Control flow costs 14–26% on every kernel, and target misalignment alone costs 2.2–8.4% while needing no hardware change at all. tarfind is the exception, dominated by divide latency.

TABLE I

Model versus RTL on seven Embench-IoT kernels at LOCAL_SCALE_FACTOR=1. The model under-predicts slightly and consistently, by the few cycles of the rdcycle pair that brackets the measured region.

kernel	instructions	CV32E40X			CV32E40P		
		model	RTL	error	model	RTL	error
matmult-int	97,748	134,543	134,545	0.00%	134,543	134,547	0.00%
primecount	2,273,871	3,927,222	3,927,224	0.00%	3,927,222	3,927,226	0.00%
edn	49,365	65,177	65,179	0.00%	65,177	65,181	0.01%
tarfind	53,846	117,147	117,149	0.00%	117,147	117,151	0.00%
md5sum	52,607	71,398	71,400	0.00%	71,398	71,402	0.01%
statemate	1,159	1,604	1,606	0.12%	1,604	1,608	0.25%
crc32	24,608	29,735	29,737	0.01%	29,735	29,739	0.01%

TABLE II

Predicted versus measured effect of an ISA change. The Zbb row is cycles saved; the Xpulp rows are speedup.

change	core	predicted	measured	err
Zbb enabled	40X	3,570	3,669	-2.7%
Xpulp, 1 loop	40P	3.243×	3.241×	-0.10%
Xpulp, matmult	40P	1.305×	1.502×	+15%

internal wires and cost nothing. Survivors are then scored with the same model, credited with the cycles the sequence costs and debited the cycles their replacement will itself take.

III. Validation

Ground truth is Verilator [6] simulation of each core through core-v-verif, with the measured region bracketed by rdcycle.

Table I gives 14 points at worst 0.25%, though that number only means something alongside how we reached it. For most of the project crc32 sat at +3.45% on both cores, and identical error on two different microarchitectures pointed at shared logic rather than either pipeline. Instruction length was being inferred from the printed width of the hex encoding, but spike -l zero-pads compressed encodings to eight digits too, so every compressed instruction was mis-sized; crc32 branches to one at a misaligned address and was charged a cycle it never paid on all 1,023 iterations. Taking length from the encoding's low two bits, as RISC-V defines, fixed it. We only found it because a newly added kernel produced a 9% error too large to write off, which is how latent modelling bugs tend to surface.

IV. Predicting a change before building it

Zbb is predicted from a baseline trace before the bitmanip hardware is enabled. Xpulp is harder: Spike cannot execute Xpulp at all, so the extension's binary cannot be traced. We instead transform the baseline trace analytically, converting counted loops to hardware loops, folding address arithmetic into post-increment load/store and fusing mul+add into p.mac, then re-cost the result with the validated model.

We have kept the +15% outlier rather than dropping it, because the reason it is wrong is the useful part. Recompiling for an extension relays out the code, and taken-branch cost depends on target alignment, which the predictor cannot know from the baseline layout; it therefore emits an explicit uncertainty band rather than false precision. More fundamentally, Xpulp's largest win on matmult is not fusion but induction-variable strength reduction: GCC restructures index-based addressing into pointer-walking because post-increment exists. Predicting an ISA extension therefore means modelling the compiler's response to it, not only the hardware's. That is a real bound on how far a single baseline trace can be pushed.

V. Discovering New Instructions

A candidate finder mines the trace for straight-line sequences worth fusing, constrained to one live-out result and a bounded number of register-file sources, and scores each with the same validated model. Four discovered instructions were added to CV32E40P by decoder edit and measured. All four computed correctly and every measured saving fell inside its predicted band. On the idiom kernels they were mined from, one instruction recovered 2,048 of 13,326 cycles for a crc32 table index (15.4%), 2,048 of 16,395 for an md5sum variable rotate (12.5%), 1,024 of 10,253 for a matmult-int indexed load (10.0%), and nothing at all for an edn fixed shift.

The bands are not hedging. Fusing removes instructions, and with them the scheduling slack that was hiding a load-use hazard: after recompiling, GCC rescheduled the shorter loop so a just-loaded value fed directly into the fused instruction, costing back what the fusion won. One instruction won its full predicted 2,048 cycles and another won zero, by the same mechanism at different strengths. A point estimate would have been wrong in both directions.

The interface, not the arithmetic, sets the granularity. We implemented pg.mac, a fused multiply-accumulate, as a CV-X-IF coprocessor [5] on CV32E40X. It computes correctly and won zero cycles: 13,321 versus 13,321. The offload costs a cycle, because the result must be registered and handed back over result_valid/result_ready, so the fused instruction retires in two cycles where mul; add took two. A two-instruction fusion across CV-X-IF can never

win; a sequence must be at least three cycles before offload pays. Feeding the measured two-cycle latency back into the finder pruned 33 nominally viable candidates on matmult-int to 11, all memory-addressing sequences, which is what Xpulp implements, now derived from measurement rather than assumed. The same operation in the pipeline (p.mac) costs one cycle, identical to add. This is the clearest evidence for why Xpulp lives in the decoder and why CV-X-IF suits coarse-grained offload instead.

Free cycles with no hardware. Branch-target misalignment costs 2.2–8.4% of all cycles across our kernels. On edn that is 4,465 cycles, larger than the 3,669 cycles won by enabling the entire bitmanip extension. All it needs is alignment from the compiler or linker.

VI. Conclusions and Future Work

A cycle model taken from published pipeline documentation rather than fitted to measurements predicted real-core cycle counts on two OpenHW cores to within 0.25%, and that was accurate enough to settle design questions before the hardware existed: whether to enable Zbb, what Xpulp would be worth, and which four custom instructions to add. Those instructions recovered 10–15% of their kernels’ cycles wherever they paid at all, and Xpulp reached $3.24\times$ on a single loop. The result we did not anticipate is that the interface, not the arithmetic, sets the smallest fusion worth building. Two upstream bugs were found and patched on the way, one leaving every CV32E40X load and store carrying zero in core-v-verif’s Verilator wrapper, the other mis-ordering nested hardware-loop registers in the PULP GCC fork.

This remains work in progress. The model covers in-order pipelines with a zero-wait-state memory; extending it to a cached core is ongoing. The granularity result above also points at its own next step: since a sequence must exceed roughly three cycles before CV-X-IF offload pays, the candidates the finder retains after calibration are exactly the coarse ones, and the natural target for them is a PULP-

style accelerator such as RedMulE [7], which attaches over CV-X-IF itself. Routing a candidate to an accelerator rather than an instruction requires predicting the residual cost left on the core (data marshalling, configuration and synchronisation), which is the same quantity the stall breakdown already reports. Code, patches and every number above: <https://github.com/ashuthosh-mr/mouna>.

Acknowledgment

The cycle models, the candidate finder, the custom-instruction RTL and the drafting of this paper were produced with the assistance of an AI coding assistant (Anthropic Claude), directed by the author. Every performance figure reported here was measured by the author on RTL simulation of the cores named, and the author is solely responsible for the content and for any errors.

References

- [1] J. Bennett et al., “Embench: A Modern Embedded Benchmark Suite,” <https://www.embench.org>.
- [2] E. Rezunov, N. Zurstraßen, L. M. Reimann and R. Leupers, “Automatic Microarchitecture-Aware Custom Instruction Design for RISC-V Processors,” arXiv:2509.15782, 2025.
- [3] OpenHW Group, CV32E40X User Manual.
- [4] OpenHW Group, CV32E40P User Manual.
- [5] OpenHW Group, CORE-V eXtension Interface (CV-X-IF) Specification.
- [6] W. Snyder, “Verilator,” <https://verilator.org>.
- [7] Y. Tortorella, L. Bertaccini, L. Benini, D. Rossi and F. Conti, “RedMulE: A Mixed-Precision Matrix-Matrix Operation Engine for Flexible and Energy-Efficient On-Chip Linear Algebra and TinyML Training Acceleration,” arXiv:2301.03904, 2023.
- [8] M. Gautschi et al., “Near-Threshold RISC-V Core With DSP Extensions for Scalable IoT Endpoint Devices,” IEEE Trans. VLSI Syst., 2017.
- [9] Spoorthi S., M. R. Ashuthosh, Bharadhwaj K., V. Reddy and M. Purnaprajna, “PARISCV: A RISC-V Profiler for Application-Specific Hardware Optimization,” in Proc. IEEE 32nd Int. Conf. High Performance Computing, Data and Analytics Workshop (HiPCW), 2025, pp. 303–304.